

Практическая работа 11

SQLITE, JDBC И ПАРАМЕТРИЗОВАННЫЕ ЗАПРОСЫ

Цель работы

Заменить адаптер памяти на SQLite через JDBC, использовать PreparedStatement и подтвердить сохранение данных после повторного открытия.

Описание программы

SqliteNutritionPlanRepository реализует прежний порт. При старте создаётся схема с ограничениями, а CRUD-операции выполняются параметризованными запросами.

Общие сведения

SQLite хранит данные в локальном файле. JDBC управляет Connection, PreparedStatement и ResultSet; значения пользователя передаются параметрами, а не склеиваются с SQL.

Функциональное назначение и ограничения

Планы сохраняются между запусками и читаются тем же сервисом. Ограничение уникальности превращается в понятный предметный отказ.

Решение локальное и однопользовательское. Схема ограничивает обязательные поля, допустимые состояния и уникальность нормализованного названия.

Логическая структура

Репозиторий открывает соединение, создаёт таблицу, подготавливает SQL, устанавливает параметры и преобразует строки ResultSet в NutritionPlan. Ресурсы закрываются try-with-resources.

Используемые технические средства

SQLite, JDBC, sqlite-jdbc, PreparedStatement, Gradle и интеграционные тесты с временной БД.

Входные и выходные данные

Входные данные. Предметный объект и путь к файлу БД; SQL-параметры.

Выходные данные. Сохранённые строки, восстановленные объекты либо предметная ошибка ограничения.

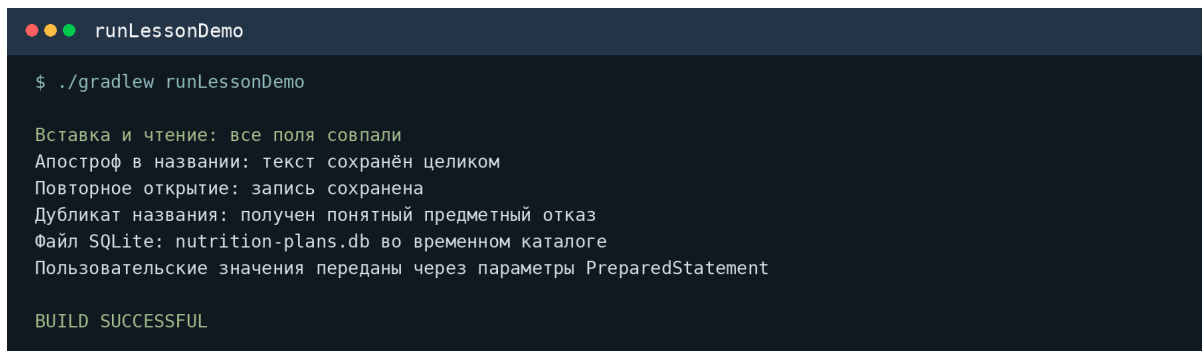
Контрольные сценарии

Компонент	Ответственность	Проверка
Пользовательский интерфейс	Ввод и обратная связь	UI-снимки
Прикладная логика	Сценарии и правила	Автотесты
Данные/инфраструктура	Состояние и хранение	Протокол запуска

Ход работы

Проект собран и проверен по командам из README и протокола проверки. Ниже приведены фактические фрагменты реализации и результаты запусков; учебные изображения преподавателя не использовались.

1. Проверка БД. Фактическое состояние получено при запуске проекта и сохранено в каталоге docs/evidence. Оно подтверждает соответствующий пользовательский сценарий и отсутствие аварийного завершения.



```
runLessonDemo
$ ./gradlew runLessonDemo

Вставка и чтение: все поля совпали
Апостроф в названии: текст сохранён целиком
Повторное открытие: запись сохранена
Дубликат названия: получен понятный предметный отказ
Файл SQLite: nutrition-plans.db во временном каталоге
Пользовательские значения переданы через параметры PreparedStatement

BUILD SUCCESSFUL
```

Рисунок 1 – JDBC, повторное открытие и ограничения

2. Начальное окно. Фактическое состояние получено при запуске проекта и сохранено в каталоге docs/evidence. Оно подтверждает соответствующий пользовательский сценарий и отсутствие аварийного завершения.

Планов: 3

[illegible]

2026-12-01

Черновик ▼

Сохранить план

Удалить выбранный

План «Снижение массы» теперь действует

Вывод

Порт репозитория реализован на SQLite без изменения UI и домена. PreparedStatement безопасно передаёт значения, данные переживают повторное открытие, а ограничения схемы поддерживают инварианты.

Выполнил: студент группы 090302-ИСТа-023	Карионов В.В.
Проверил: преподаватель	Елкин Н.С.